RICE UNIVERSITY

Graphs and Transformations

From one op at a time to whole-program compilation: torch.compile and JAX

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 9, Lecture 2

Where we are

Lecture 1 found	**Today's response**
Each op pays Python + dispatch + launch	Capture many ops as **one graph**
Each op reads and writes HBM	**Fuse** ops: fewer bytes, fewer launches
Autograd records the graph at run time	Build the backward graph **ahead of time**
Our MLP is overhead-bound	Measure eager vs compiled vs JAX

In our pipeline

T1: compare **PyTorch eager**, **torch.compile**, and a guided **JAX** version; separate **compile** from **steady-state** cost.

Learning goals for today

- Explain what **eager** execution cannot do, and why a **graph** helps (fusion, fewer launches).
- Trace **torch.compile**: Dynamo $\rightarrow$ FX graph $\rightarrow$ AOTAutograd $\rightarrow$ Inductor $\rightarrow$ Triton kernels.
- Recognize **graph breaks** and **recompilation**, and fix the common causes.
- Write the forward pass in JAX and use **vmap** and **jit** as transformations of pure functions.
- Time execution modes fairly and compute when compilation **pays for itself**.

A puzzle to start

Eager
400 µs per step
no compile

Compiled
150 µs per step
40 s to compile, once

After how many training steps does compiling pay off?

Key idea

40 s ÷ 250 µs = **160,000 steps**. A 1,000-step run is **slower** with compilation; a million-step run is much faster.

Part 1

Eager execution and its costs

What does executing one op at a time leave on the table?

Eager mode: define by run

```
t1 = a * x            # launch kernel 1
t2 = t1 + b           # launch kernel 2
y = torch.tanh(t2)    # launch kernel 3
print(y[0])           # real values, now
```

- Each line **runs immediately**
- Results are real tensors
- Any Python works: `if`, loops, `print`

Watch out

Eager sees **one op at a time**: it cannot know what comes next, so it cannot combine ops.

What one-op-at-a-time costs

Eager: 3 kernels

$t_1 = a \cdot x$ write t_1

$t_2 = t_1 + b$ read t_1, write t_2

$y = \tanh(t_2)$ read t_2, write y

Q = 24n bytes, 3 launches

Fused: 1 kernel

$y = \tanh(a \cdot x + b)$

read x, write y

t_1, t_2 stay in registers

Q = 8n bytes, 1 launch

Key idea

Fusion needs to see **several ops at once** — that is, a **graph**.

Source: Week 8, Lecture 1 (fusion and the roofline).

A cost model for one training step

$$T(\text{step}) \approx \Sigma \max(\ t(\text{issue}), t(\text{kernel})\ )$$

$$\text{small kernels:}\ \ T(\text{step}) \approx N \cdot t(\text{issue})$$

$$\text{our MLP:}\ \ \approx 25 \times \approx 4\ \mu\text{s} \approx 100\ \mu\text{s}$$

Two levers

Fewer kernels N: fuse.

Cheaper issue t(issue): capture once, replay (CUDA graphs).

In our pipeline

Both levers need the program **ahead of time** — as a graph, not one op at a time.

Graph capture: the payoffs

With a graph, a compiler can...	Which removes...
Fuse elementwise ops and reductions	HBM round trips, launches
Plan buffer reuse	allocations, peak memory
Specialize to shapes and dtypes	generic-kernel overhead
Replay a kernel sequence (CUDA graphs)	per-launch CPU cost
Plan backward: save or recompute	activation memory

Watch out

The price: **compile time**, and limits on the Python you can write. Parts 2–4 are about both.

Check yourself

Two minutes with a neighbour.

- Four elementwise ops in a row on n FP32 values: bytes moved eager vs fused?
- 40 kernels, each 0.25 µs on the GPU, issue cost 5 µs each. Estimate the step time.
- Why does fusion help a chain of elementwise ops much more than a large matmul?

Part 2

Capturing a graph: torch.compile

How do you get a graph out of ordinary Python?

Two ways to capture a graph

	Tracing	**Python-level capture (Dynamo)**
How	run once, record tensor ops	read bytecode as it runs
Python `if` on a tensor	baked in: one branch	graph break, still correct
Validity check	none: user's problem	**guards** checked on every call
Examples	`torch.jit.trace`	`torch.compile`

Key idea

Dynamo's promise: **never wrong**, only sometimes **less compiled**.

Source: PyTorch documentation: torch.compile, "torch.compiler" overview, TorchDynamo and Inductor (pytorch.org/docs).

Tracing bakes in control flow

```
def g(x):
    if x.sum() > 0:
        return x * 2
    return x - 1

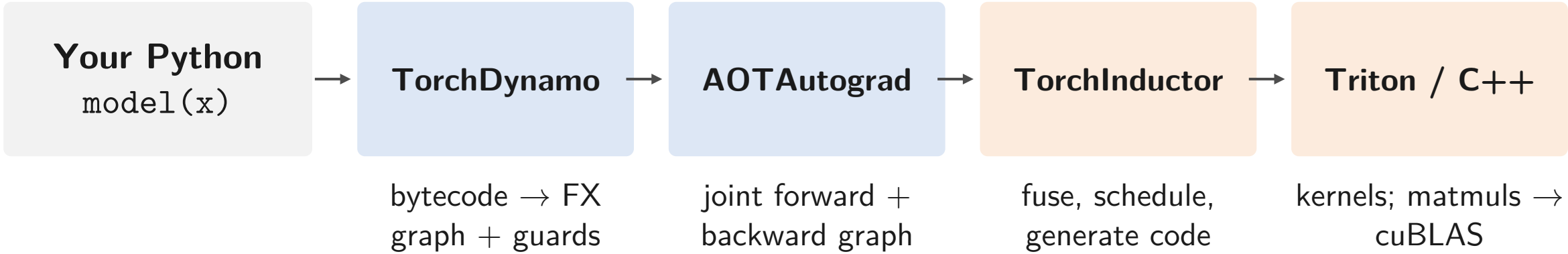
tg = torch.jit.trace(g, torch.ones(3))
tg(-torch.full((3,), 5.))   # gives -10, not -6
```

- Recorded: **only** `x * 2`
- The `if` ran once, at trace time
- Wrong inputs give **wrong answers**, not errors

Watch out

A tracer sees **values**, not **code**: control flow on tensor values is frozen.

The torch.compile pipeline



Key idea

Compiled code is **cached** and reused while the **guards** hold; otherwise Dynamo recompiles.

Source: PyTorch documentation: torch.compile, "torch.compiler" overview, TorchDynamo and Inductor (pytorch.org/docs).

What Dynamo captures

```
def show(gm, example_inputs):
    print(gm.code)
    return gm.forward

@torch.compile(backend=show)
def f(x, a, b):
    return torch.tanh(a * x + b)

f(torch.randn(8), 2.0, torch.randn(8))
```

printed (simplified)

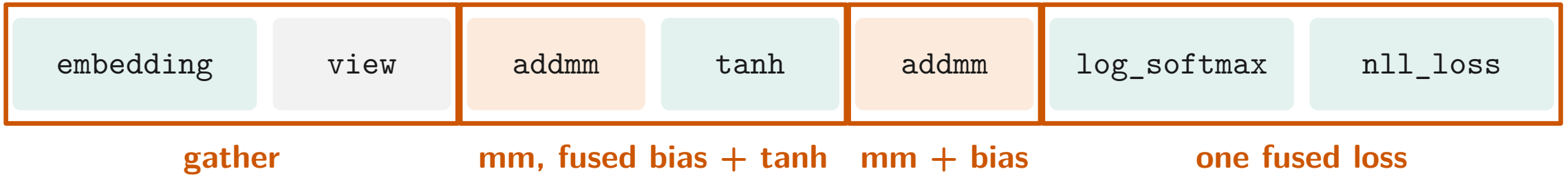
```
def forward(self, L_x_, L_b_):
    mul = 2.0 * L_x_
    add = mul + L_b_
    tanh = torch.tanh(add)
    return (tanh,)
```

- `a = 2.0` became a **constant**, with a guard
- A straight-line graph: ready to **fuse**

Source: PyTorch documentation: torch.compile, "torch.compiler" overview, TorchDynamo and Inductor (pytorch.org/docs).

Inductor: from graph to fused kernels

eager forward: 7 ops, 6 kernels (view is free)



compiled: fewer, fused kernels (schematic)

Key idea

Fewer kernels → fewer **launches** and fewer **HBM round trips**. Read the generated Triton with `TORCH_LOGS="output_code"`.

Source: PyTorch documentation: torch.compile, "torch.compiler" overview, TorchDynamo and Inductor (pytorch.org/docs). Grouping is schematic.

Using torch.compile

```
model = MLP().cuda()
cmodel = torch.compile(model)    # nothing yet

for x, y in loader:
    loss = F.cross_entropy(cmodel(x), y)
    loss.backward()    # compiled backward
    opt.step(); opt.zero_grad()
```

mode=	Adds
"default"	fusion, Triton
"reduce-overhead"	CUDA graphs
"max-autotune"	matmul tuning

Watch out

The **first call** compiles (seconds or more); later calls reuse the cached code.

Source: PyTorch documentation: torch.compile, "torch.compiler" overview, TorchDynamo and Inductor (pytorch.org/docs).

Check yourself

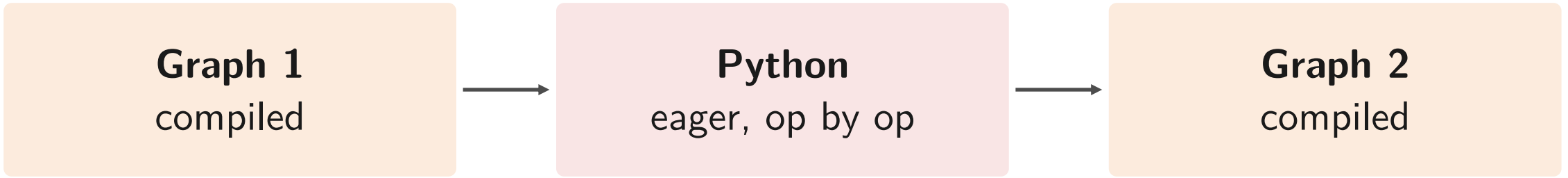
- Name the three stages of torch.compile and one job each does.
- What does `torch.jit.trace(g, torch.ones(3))` return for an input with negative sum?
- `f(x, 2.0, b)` is compiled. Then you call `f(x, 3.0, b)`. What happens?

Part 3

Graph breaks and recompilation

When does compilation quietly give up, or start over?

What a graph break is



- Still **correct**
- No fusion **across** the break; intermediates go to HBM
- Extra launches and Python overhead in the middle

Watch out

One break in the hot loop can undo most of the speedup.

Source: PyTorch documentation: torch.compile, "torch.compiler" overview, TorchDynamo and Inductor (pytorch.org/docs).

Three common causes

```
if y.sum() > 0:
```

branch on a tensor value — 2 graphs, 1 break

```
s = y.sum().item()
```

tensor → Python number — 2 graphs, 1 break

```
print("loss", y)
```

Python side effect — 2 graphs, 1 break

Key idea

All three need a **concrete value** or a **Python side effect** in the middle of the graph.

Source: torch._dynamo.explain on each example (PyTorch 2.x, run on CPU).

Finding and fixing graph breaks

```
e = torch._dynamo.explain(f)(x)
e.graph_count, e.graph_break_count    # (2, 1)
e.break_reasons
```

Cause	Fix
`if y.sum() > 0:`	`torch.where(...)` or `torch.cond(...)`
`.item()` in the step	keep it a tensor; log outside, every k steps
`print` in the step	move out of the compiled function

Check

Find: `explain`, `TORCH_LOGS="graph_breaks"`. Enforce: `fullgraph=True`.

Source: PyTorch documentation: torch.compile, "torch.compiler" overview, TorchDynamo and Inductor (pytorch.org/docs).

Guards and recompilation

call 1: $B = 64$	→	compile
call 2: $B = 64$	→	cache hit
call 3: $B = 32$	→	guard fails → recompile, B now dynamic
call 4: $B = 16$	→	cache hit (dynamic)

TORCH_LOGS="recompiles"

```
[__recompiles] Recompiling function forward
  triggered by the following guard failure(s):
  - tensor 'x' size mismatch at index 0.
    expected 64, actual 32
```

Watch out

Limit: **8** recompiles per function, then eager.

Tip: `drop_last=True`.

Source: observed with `TORCH_LOGS="recompiles"` (PyTorch 2.x); default `recompile_limit` from the torch.compile docs.

Check yourself

- Your compiled step ends with `losses.append(loss.item())`. What does that cost, and the fix?
- Calls with batch 64, then 128, then 100. How many compilations?
- Why might you turn on `fullgraph=True` while developing?

Part 4

JAX: function transformations

What if the whole framework were built on graphs?

The functional model

	PyTorch (eager)	JAX
Parameters	inside `nn.Module`, mutated	explicit pytree argument
Arrays	mutable, in-place ops	immutable: `x.at[i].set(v)`
Gradient	`loss.backward()` fills `.grad`	`grad(loss)` returns a function
Randomness	global generator	explicit PRNG keys

Key idea

Pure functions can be **transformed**: `vmap`, `jit` — and **composed**, e.g. `jit(vmap(f))`.

Source: JAX documentation: Automatic differentiation, Automatic vectorization, Just-in-time compilation (docs.jax.dev).

Our MLP's forward pass in JAX

```
def forward(p, x):                  # x: (B, 8) ids
    e = p["E"][x].reshape(len(x), -1)
    h = jnp.tanh(e @ p["W1"] + p["b1"])
    return h @ p["W2"] + p["b2"]    # logits

forward(params, ids).shape          # (64, 322)
```

- **Pure**: output depends only on `p` and `x`.
- Parameters: an explicit **pytree**.
- Same 34,338 parameters as PyTorch.

Key idea

A pure forward function is what `jit` and `vmap` can **transform**.

Source: JAX documentation: Automatic differentiation, Automatic vectorization, Just-in-time compilation (docs.jax.dev).

What JAX sees: the jaxpr

```
jax.make_jaxpr(f)(1.0)
# { lambda ; a:f32[]. let
#     b:f32[] = sin a
#     c:f32[] = mul a b
#   in (c,) }
```

```
layer = lambda w, x: jnp.tanh(x @ w)
jax.make_jaxpr(layer)(w, x)
# { lambda ; a:f32[3,2] b:f32[4,3]. let
#     c:f32[4,2] = dot_general[...] b a
#     d:f32[4,2] = tanh c
#   in (d,) }
```

Key idea

The forward pass is a **typed, straight-line graph** of primitives — ready to compile and fuse.

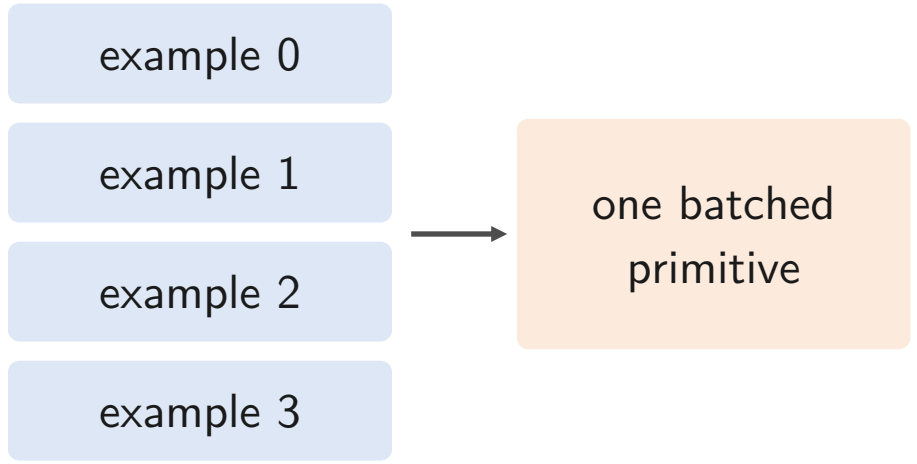
Source: JAX documentation: Automatic differentiation, Automatic vectorization, Just-in-time compilation (docs.jax.dev).

vmap: batching without loops

```
def dot(a, b):                # one example
    return jnp.dot(a, b)

bdot = jax.vmap(dot)          # a batch: (n, d)
bdot(A, B).shape              # (n,)

one = lambda p, x1: forward(p, x1[None])[0]
bfwd = jax.vmap(one, in_axes=(None, 0))
bfwd(params, ids).shape       # (64, 322)
```



Key idea

Write **one example**; `vmap` gives the **batched** forward pass.

Source: JAX documentation: Automatic differentiation, Automatic vectorization, Just-in-time compilation (docs.jax.dev).

jit: tracing with abstract values

```
@jax.jit
def g(x):
    print("tracing", x)   # runs at trace time
    return x * 2

g(jnp.ones(3))    # trace + compile: float32[3]
g(jnp.ones(3))    # cached: no print
g(jnp.ones(4))    # retrace: float32[4]
g(jnp.ones(3, dtype=jnp.int32))  # retrace
```

- Tracer = **shape** + **dtype**, no values
- Cache key: shapes and dtypes
- New shape or dtype → **retrace**

Watch out

Python side effects (print, appending to a list) run **once per trace**, not per call.

Source: JAX documentation: Automatic differentiation, Automatic vectorization, Just-in-time compilation (docs.jax.dev).

Control flow under jit

```
@jax.jit
def h(x):
    if x.sum() > 0:  # no value
        return x
    return -x

h(jnp.ones(3))
# TracerBoolConversionError
```

Fix	Use when
`jnp.where(c, a, b)`	both sides cheap
`jax.lax.cond(...)`	a real branch
`static_argnums=`	known before the call

Key idea

torch.compile **breaks the graph and continues**; jax.jit **raises**. Same root cause: no value at trace time.

Source: JAX documentation: Automatic differentiation, Automatic vectorization, Just-in-time compilation (docs.jax.dev).

Asynchronous dispatch

```
mm = jax.jit(lambda a: a @ a)
mm(M).block_until_ready()      # warm up

t0 = time.perf_counter()
r = mm(M)                      # returns at once
t1 = time.perf_counter()       # 0.06 ms
r.block_until_ready()          # wait for result
t2 = time.perf_counter()       # 32 ms
```

returned: 0.06 ms

finished: 32 ms

2000 × 2000 matmul, laptop CPU

Watch out

Always **block** before reading the clock: `.block_until_ready()` in JAX, `torch.cuda.synchronize()` in PyTorch.

Source: JAX documentation, "Asynchronous dispatch"; timing measured on a laptop CPU.

torch.compile and jax.jit, side by side

	torch.compile	**jax.jit**
Captures by	reading bytecode (Dynamo)	tracing with abstract values
Unsupported Python	graph break, runs eagerly	error
New input shape	guard fails $\rightarrow$ recompile, then dynamic	retrace + recompile
`print` inside	graph break	runs once, at trace time
Code generator	Inductor $\rightarrow$ Triton	XLA
Mutation	allowed	arrays immutable

Key idea

Same goal — a compiled graph — from opposite ends: **flexible first** vs **functional first**.

Source: PyTorch documentation, torch.compile; JAX documentation, Just-in-time compilation.

Check yourself

- A jitted `g` with a `print` is called on shapes (8,), (16,), (8,). How many prints?
- `t0 = time(); y = step(p, x); t1 = time()` gives 50 µs. Why is that wrong?
- `one(params, x1)` computes logits for one example. Batch it over `X` with `vmap`.

IN-CLASS BUILD

Compare execution modes

Work in pairs. Separate compile cost from step cost.

Exercise: three ways to run one step

Mode (MLP, B = 64, SGD)	First call	Steady step	Compile cost
PyTorch eager	0.3 s	400 µs	—
`torch.compile(model)`	30 s	160 µs	?
JAX `jax.jit(step)`	1.5 s	120 µs	?

practice numbers, not measurements

DO

A. What must happen before you stop the clock? What counts as compile cost?

B. After how many steps does each compiled mode beat eager?

STRETCH

Why might the first **eager** call also be slow?

Timing PyTorch fairly

```
def timed(fn):
    torch.cuda.synchronize()
    t0 = time.perf_counter(); fn()
    torch.cuda.synchronize()
    return time.perf_counter() - t0

cm = torch.compile(model)   # lazy: no work yet
run = lambda: step(cm, x, y)
first = timed(run)          # compile + 1 step
steady = statistics.median(
    timed(run) for _ in range(100))
```

- **First call** = compile + one step
- **Steady** = median of later steps
- **Synchronize** around every timing
- Recompile: `torch._dynamo.reset()`

Source: PyTorch documentation: torch.compile, torch.cuda.synchronize.

The JAX version

```
def loss_fn(p, x, y):
    e = p["E"][x].reshape(len(x), -1)
    h = jnp.tanh(e @ p["W1"] + p["b1"])
    z = h @ p["W2"] + p["b2"]
    lp = jax.nn.log_softmax(z)
    lp = jnp.take_along_axis(lp, y[:, None], 1)
    return -lp.mean()

sgd = lambda w, d: w - 0.1 * d
@jax.jit
def step(p, x, y):
    l, g = jax.value_and_grad(loss_fn)(p, x, y)
    return jax.tree.map(sgd, p, g), l
```

```
# first call:
p, l = step(p, x, y)
l.block_until_ready()
```

- Forward, backward, update: **one** XLA program
- Same parameters: **34,338**

Source: JAX documentation: Automatic differentiation, Automatic vectorization, Just-in-time compilation (docs.jax.dev).

When does compilation pay off?

$$T(N) = t(\text{first}) + (N - 1) \cdot t(\text{step})$$

$$\text{compiled beats eager when } \; T(\text{compiled}, N) < T(\text{eager}, N)$$

$$N^* = 1 + \Delta t(\text{first}) \; / \; \Delta t(\text{step})$$

Δt(first): extra first-call time of the compiled mode · Δt(step): its saving per step

Compute

Break-even = **extra one-time cost ÷ saving per step**. Report both numbers, not just the speedup.

Exercise: solutions

Mode	Extra first-call cost	Saving per step	Break-even N*
`torch.compile`	$30 - 0.3 = 29.7$ s	240 µs	$\approx$ **123,751**
JAX `jit`	$1.5 - 0.3 = 1.2$ s	280 µs	$\approx$ **4,287**

- A: **synchronize / block** before stopping the clock; compile cost = first call − one step.
- Stretch: the first eager call pays CUDA context, library loading, first allocations.

Check

A 1,000-step run: eager wins against torch.compile here; JAX's cheaper compile pays off after $\approx$ 4,300 steps.

What goes into T1

T1 part	From
Triton kernel validated against PyTorch	Week 8
Memory traffic and arithmetic intensity predictions	Week 8
Eager vs compiled vs JAX: first call and steady state	**Today**
Compile, launch, and steady-state costs separated	Weeks 9–10

In our pipeline

Report **graph breaks and recompiles** you found, and the **break-even** step count.

Key takeaways

- Eager runs **one op at a time**: per-op overhead, no fusion.
- A **graph** enables fusion, fewer launches, memory planning, CUDA graphs.
- torch.compile: **Dynamo** $\rightarrow$ FX graph + guards $\rightarrow$ **AOTAutograd** $\rightarrow$ **Inductor** $\rightarrow$ Triton.
- **Graph breaks** (tensor-valued `if`, `.item()`, `print`) and **recompiles** silently cost speed.
- JAX: pure forward functions; `vmap` / `jit` as **composable transformations**.
- Time with **synchronization**; break-even = one-time cost $\div$ per-step saving.

Next time, and readings

When	What
Before spring break	T1 checkpoint due
Lab 3	Time eager, compiled, and JAX steps; find graph breaks
Week 10, Lecture 1	Framework and GPU interaction: fusion, memory, precision

READINGS (FOR LAB 3)

PyTorch docs: *torch.compile* tutorial and *torch.compiler* troubleshooting.

JAX docs: *Just-in-time compilation* and *Asynchronous dispatch*.